ENGR 102 Summer Bridge
Day 12 Instructor Key

For Loops, Range, Counters, and Accumulators

Tuesday, July 21, 2026 • 50 points

Name		UIN		Section	

Boolean-operator convention. Python operators appear as `and`, `or`, and `not`. Their lowercase spelling is required in code.

Daily target — increasing independence

Build fixed-count programs that preserve loop state and exact bounds.

Allowed tools	Day 10 tools plus for, range, counters, and accumulators
Support supplied	Required inputs, required outputs, and one sample run are supplied.
You must decide	Initialization, range bounds, accumulator updates, and the placement of the threshold decision.
Scaffold level	State inventory prompted once; loop body is student-authored.

Scoring and completion standard — 50 points

Evidence	Points	Secure work shows
Integrated trace	10	intermediate state, condition results, and exact output
Diagnosis and repair	10	actual, intended, cause, repair, and a boundary test when requested
Full program and tests	30	coherent executable logic, required output, and defended tests

Independence standard. You may use the supplied requirements and examples, but you must produce one connected solution. A collection of disconnected correct lines is not yet a complete program.

Begin with the trace, then use its evidence habits when you construct.

Task 1. Integrated trace — 10 points

Trace the range, the conditional counter, and the accumulator after every pass.

```
total = 0
multiple_count = 0

for number in range(2, 11, 2):
    if number % 4 == 0:
        multiple_count = multiple_count + 1
    total = total + number - multiple_count
    print(number, total, multiple_count)
```

Your task

1. List the values produced by range(2, 11, 2).
2. Create one pass ledger containing number, multiple_count after the if, and total after the update.
3. Write all printed lines in exact order.

Answer and explanation

1. The range values are 2, 4, 6, 8, 10. The stop value 11 is excluded.
2. Pass states (number, multiple_count, total): (2, 0, 2), (4, 1, 5), (6, 1, 10), (8, 2, 16), (10, 2, 24). The counter update occurs before that pass contributes to total.
3. Exact output lines: 2 2 0; 4 5 1; 6 10 1; 8 16 2; 10 24 2.

A final answer without the requested state evidence cannot earn full trace credit.

Task 2. Diagnose and repair — 10 points

The intended program must add the integers from 1 through limit, inclusive.

```
limit = int(input())
total = 0

for number in range(1, limit):
    total = total + number

print(total)
```

Your task

1. For limit = 4, state the actual and intended totals and identify the exact cause.
2. Write the minimal repair and a boundary test that would expose the original bug.

Answer and explanation

1. range(1, 4) produces 1, 2, 3, so the actual total is 6. The intended inclusive total is $1 + 2 + 3 + 4 = 10$. The loop excludes limit because range excludes its stop value.
2. Minimal repair: for number in range(1, limit + 1). A strong boundary test is limit = 1: the buggy loop totals 0, but the repaired loop totals 1.

Debugging evidence is complete only when the repair is tied to the stated defect and an expected result.

Task 3. Construct a complete program — 30 points across Pages 4-5

Program specification

- Read sample_count as an integer and threshold as a float.
- If sample_count is not positive, print Invalid count and do not ask for readings.
- Otherwise read exactly sample_count floating-point readings.
- Compute the total, average, and number of readings at or above threshold.
- Print total, average, and at_or_above on separate lines in that order.

Required planning evidence

1. Name the state that must exist before the loop and give each initial value.

Answer and explanation

```
sample_count = int(input())
threshold = float(input())

if sample_count <= 0:
    print("Invalid count")
else:
    total = 0.0
    at_or_above = 0

    for sample_number in range(sample_count):
        reading = float(input())
        total = total + reading
        if reading >= threshold:
            at_or_above = at_or_above + 1

    average = total / sample_count
    print(total)
    print(average)
    print(at_or_above)
```

total and at_or_above must be initialized once before repetition. The loop runs exactly sample_count times because range(sample_count) produces that many indexes.

The division occurs after the loop and only in the positive-count branch, so division by zero is impossible.

Task 3 continued. Test and defend the program **included in 30 points**

Expected tests and results

1. Count 4, threshold 4.0, readings 3.5, 4.0, 2.5, 5.0 prints 15.0, 3.75, 2 on separate lines.
2. Count 0 and any threshold prints Invalid count and requests no readings.

Scoring guidance

4 points state plan; 4 input/guard; 6 correct loop bound and repeated input; 5 total; 4 threshold count; 4 average/output; 3 tests.

Accept alternative correct code when it obeys the stated tool boundary, handles the required cases, and produces the required output. All blue text is answer or scoring guidance.

VIP Lab Alignment

Bridge Day 12 • Contract 2d4127aac856

This page adds a current lab handoff while preserving the workbook's independence-ramp tasks.

Why this page is here

VIP Lab Day(s): 5 **Activities:** 18.18, 18.20

Use deliberate range bounds and comparison state to collect multiples and preserve the smallest value.

Open these current notebook cells

Activity / stable cells	Notebook work	Evidence to carry forward
18.18 18-18-work / 18-18-transfer / 18-18-cold	**Multiples In Range**	Choose inclusive range bounds.
18.20 18-20-work / 18-20-transfer / 18-20-cold	**Smallest Value**	Initialize comparison state from real data.

Readiness check before you code

1. Write the first value, final included value, and excluded stop value for the range used in Activity 18.18.

   Answer and explanation: The exact values depend on the sample or transfer inputs, but the final desired endpoint must be included and Python's range stop must be one step beyond it when the step is positive.

2. Explain why Activity 18.20 initializes from real data instead of from zero.

   Answer and explanation: Zero is not guaranteed to be in the data and can be smaller or larger than every real value. Initializing from the first data value makes the comparison valid for all nonempty inputs.

Notebook and submission procedure

1. Use the sample and transfer cells for formative practice; attempt before opening a reveal.
2. Complete the end-of-notebook Independent Program Studio without a reveal or copied solution. Only the Studio cells listed above are scored for independent mastery in the matching Lab Day notebook.
3. Save or download the completed notebook as one `.ipynb` file.
4. Upload that one notebook to the matching Gradescope assignment. Do not export a `.py` file or create a ZIP.